

OOP Group Project Reflection Report

Group 23: 陳展凱、簡才斌、鄭祐丞

Part 1 and Part 2 Overview

For part 1, we ran the code and it successfully trained, this served as our first experience with the gym environment. For part 2, we changed the three given parameters and unfortunately the accuracy is around 63%, not reaching the 70% threshold.

Part 3 Overview

For Part 3, we built a 2048 game environment from scratch, and trained a Deep Q-Network agent to control. We structured everything around the Gymnasium Env interface and split the code into separate pieces for the agent, environment, and training/testing scripts.

1. OOP concept(s) implemented in Part 3

1.1 Abstraction (Interfaces / Contracts)

Our key design goal was to define a clean and minimal contract between components.

- **Agent interface (`BaseAgent`)** (`part3/agent_2048.py`): `BaseAgent` is an abstract base class that requires every agent to implement `act()`, so training/testing code can call the same method regardless of the underlying strategy.
- **Environment interface (`Gymnasium gym.Env`)** in (`part3/env_2048.py`): Our `TwentyFortyEightEnv` follows the Gymnasium API (`reset`, `step`, `render`, `close`), which allows us to use `gym.make()` and `gym.vector.SyncVectorEnv()` without glue code.

The training loop could take advantage of abstraction with guaranteed inputs/outputs.

1.2 Inheritance (Reuse + Specialization)

We used inheritance mostly to avoid repeating code. `RandomAgent` and `DQNAgent` both inherit from `BaseAgent`, so they follow the same contract but differ in how they pick actions (one's random, one's learned). `DQNAgent` also adds `save/load` methods for checkpointing. In addition, `QNetwork` inherits from `nn.Module` and `TwentyFortyEightEnv` inherits from `gym.Env`, let us focus on the 2048-specific logic instead of reinventing wheels.

1.3 Polymorphism (Interchangeable Components)

Polymorphism appears when we swap different concrete types through the same interface. In `part3/main_2048.py`, the same `test()` function works with both `DQNAgent` and `RandomAgent` because they both have `act()`. We also made `act()` accept an optional `action_mask`, so the environment can filter out illegal moves without the training loop needing to know 2048 rules. This made it really easy to swap in a baseline agent vs. the trained one during experiments.

1.4 Encapsulation (State + Behavior bundled together)

Each class keeps its own data private and only exposes methods to interact with it. `TwentyFortyEightEnv` hides the board state (`self.board`), reward calculations, game-over checks, and rendering behind `reset/step/render`. Similarly, `ReplayBuffer` handles storing transitions and sampling batches internally—external code just calls `push()` and `sample()`. This helps prevent accidental bugs where one part of the code messes with another part's data.

1.5 Composition & Separation of Concerns (Modular design)

We broke Part 3 into separate modules:

- `env_2048.py`: game mechanics, rewards, action masking, rendering
- `agent_2048.py`: DQN agent, replay buffer, neural network
- `main_2048.py`: training/testing orchestration

This means we can tweak the reward function without touching the neural network, or swap in a different agent algorithm without rewriting the environment.

2. Implementation of Part 3

2.1 Custom 2048 Gymnasium environment

We built a custom environment (`2048-v0`) that follows Gymnasium conventions. It has a `Discrete(4)` action space (up/down/left/right) and a 4×4 grid for observations. In `step(action)`, we rotate the board so every move can reuse the same “merge left” logic (`_merge_left()`), then rotate it back. If the board changes, we add a random tile. The episode ends when `_is_game_over()` detects no empty cells and no valid merges.

2.2 Action masking (valid-move detection)

To help learning converge, we added action masking. The agent uses this mask to zero out Q-values for invalid actions by setting them to a huge negative number.

- `_get_action_mask()` simulates each action and marks it valid only if it changes the board.
- `reset()` returns `info={"action_mask": mask}` and `step()` includes it in every `info` dict.

2.3 Reward design (reward shaping)

The base reward is whatever score you get from merging tiles. We also added extra terms in `step()` to help guide the agent toward better strategy:

- **Penalty** when the board is getting too full (less than 3 empty cells).

- **Bonus** if the biggest tile is in a corner (a standard 2048 trick).

2.4 DQN agent with target network + replay buffer

In `DQNAgent` we implemented the core ideas of DQN: we store experiences as `(state, action, reward, next_state, done)` tuples in a `ReplayBuffer` and sample random batches for training. We maintain two networks, `q_net` and `target_net`; the target network is periodically synced from `q_net` to stabilize training. Exploration will gradually decrease as epsilon decays and more steps are decided by the network instead of random actions. Since 2048 tile values grow exponentially, we preprocess with `log2(max(tile, 1))` and reshape to `(batch, 1, 4, 4)` before feeding them into `QNetwork` (a small CNN followed by fully connected layers) that outputs 4 Q-values. The `QNetwork` has 2 convolutional layers to capture spatial patterns on the board, followed by fully connected layers to output Q-values for each action.

2.5 Acceleration of training

To speed up learning, we run 32 environments in parallel using `gym.vector.SyncVectorEnv`. The training loop handles batched observations and action masks, asks the agent for a batch of actions, and stores all the transitions at once. After we've collected enough data, we call `agent.update()` multiple times to do learning updates. This is way faster than running one environment at a time. Numpy arrays are also heavily utilized for speed, the game board and replay memory are both utilizing this.

2.6 Testing, baseline, rendering and results

We included `RandomAgent` as a simple baseline that picks valid moves randomly. The trained model is saved as `dqn_2048.pth` after training. During testing, we load this model and run a few episodes while rendering the game board to visually see how well the agent plays compared to random. The render is disabled for training and optional for testing to speed things up. Our final model is trained with 84 million steps, achieving an average max tile of around 1024 during testing, with occasional runs reaching 2048.

3. What we learned from Part 3

We learned that OOP in RL isn't just about making classes, it's about making sure you can swap pieces in and out easily. By using Gymnasium's standard API and a shared `BaseAgent` interface, we could compare a random baseline with DQN or try different reward functions without major rewrites. We also saw how reward shaping is a double-edged sword: it speeds up learning but can accidentally teach the agent weird behaviors. On the engineering side, things like vectorized environments and action masking made a huge difference in both speed and training stability.